

§ 11.3 KMP算法

11.3.1 构思

上一节的分析表明，蛮力算法在最坏情况下所需时间，为主串长度与模式串长度的乘积，故无法应用于规模稍大的应用环境，很有必要改进。为此，不妨从分析以上最坏情况入手。

稍加观察不难发现，问题在于这里存在大量的局部匹配：每一轮的 m 次比对中，仅最后一次可能失配。而一旦发现失配，主串、模式串的字符指针都要回退，并从头开始下一轮尝试。

实际上，这类重复的字符比对操作没有必要。既然这些字符在前一轮迭代中已经接受过比对并且成功，我们也就掌握了它们的所有信息。那么，如何利用这些信息，提高匹配算法的效率呢？

以下以蛮力算法的前一版本（代码11.1）为基础进行改进。

■ 简单示例

如图11.4所示，用 $T[i]$ 和 $P[j]$ 分别表示当前正在接受比对的一对字符。

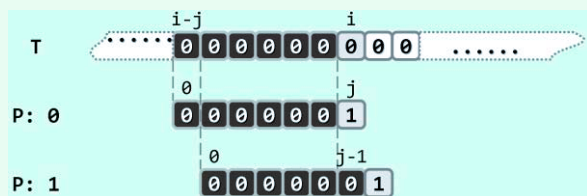


图11.4 利用以往的成功比对所提供的信息，可以避免主串字符指针的回退



图11.5 利用以往的成功比对所提供的信息，有可能使模式串大跨度地右移

当本轮比对进行到最后一对字符并发现失配后，蛮力算法会令两个字符指针同步回退（即令 $i = i - j + 1$ 和 $j = 0$ ），然后再从这一位置继续比对。然而事实上，指针 i 完全不必回退。

■ 记忆 = 经验 = 预知力

经过前一轮比对，我们已经清楚地知道，子串 $\text{substr}(T, i - j, j)$ 完全由'0'组成。记住这一性质便可预测出：在回退之后紧接着的下一轮比对中，前 $j - 1$ 次比对必然都会成功。因此，尽可令 i 保持不变、 $j = j - 1$ ，然后继续比对。如此，将使下一轮的比对减少 $j - 1$ 次！

上述“令 i 保持不变、 $j = j - 1$ ”的含义，可理解为“令 P 相对于 T 右移一个单元，然后从前一失配位置继续比对”。实际上这一技巧可推而广之：利用以往的成功比对所提供的信息（记忆），不仅可避免主串字符指针的回退，而且可使模式串尽可能大跨度地右移（经验）。

■ 一般实例

如图11.5所示，再来考查一个更具一般性的实例。

本轮比对进行到发现'E' = $T[i] \neq P[4] = 'O'$ 失配后，在保持 i 不变的同时，应将模式串 P 右移几个单元呢？有必要逐个单元地右移吗？不难看出，在这一情况下移动一个或两个单元都是徒劳的。事实上，根据此前的比对结果，必然有

$$\text{suffix}(\text{prefix}(T, i), 4) = \text{substr}(T, i - 4, 4) = \text{prefix}(P, 4) = \text{"REGR"}$$

若在此局部能够实现匹配，则至少紧邻于 $T[i]$ 左侧的若干字符均应得到匹配——比如，当 $P[0]$ 与 $T[i - 1]$ 对齐时，即属这种情况。注意到 $i - 1$ 是能够如此匹配的最左侧位置，故可放心地将 P 右移 $4 - 1 = 3$ 个单元（等效于 i 保持不变，同时令 $j = 1$ ），然后继续比对。

11.3.2 next表

一般地，如图11.6假设前一轮迭代终止于 $T[i] \neq P[j]$ 。按以上构想，指针 i 不必回退，而是将 $T[i]$ 与 $P[t]$ 对齐并开始新一轮比对。那么， t 准确地应该取作多少呢？

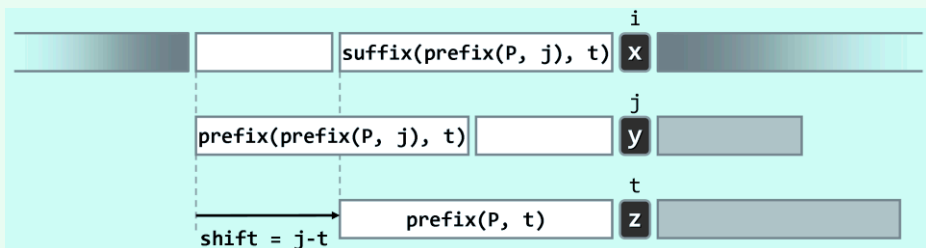


图11.6 利用此前成功比对所提供的信息，在安全的前提下尽可能大跨度地右移模式串

由图可见，经过此前一轮迭代的比对，已经确定匹配的范围应为：

$$\text{prefix}(P, j) = \text{substr}(T, i - j, j) = \text{suffix}(\text{prefix}(T, i), j)$$

于是经适当右移之后，如果模式串 P 能够与 T 的某一（包含 $T[i]$ 的）子串完全匹配，那么必要的条件之一就是：

$$\text{prefix}(P, t) = \text{prefix}(\text{prefix}(P, j), t) = \text{suffix}(\text{prefix}(P, j), t)$$

也就是说，在 $\text{prefix}(P, j)$ 中长度为 t 的真前缀应该与长度为 t 的真后缀完全匹配。更准确地，值得试探的 t 必然来自集合：

$$N(P, j) = \{ t \mid \text{prefix}(\text{prefix}(P, j), t) = \text{suffix}(\text{prefix}(P, j), t), 0 \leq t < j \}$$

需特别注意的是，集合 $N(P, j)$ 具体由哪些 t 值构成，仅取决于模式串 P 以及前一轮迭代的失配位置 j ，而与主串 T 无关！

从图11.6还可看出，若下一轮迭代从 $T[i]$ 与 $P[t]$ 的比对开始，其效果相当于将模式串 P 右移 $j - t$ 个单元。因此，为保证模式串与主串的对齐位置（指针 i ）绝不倒退，同时又不致遗漏任何可能的匹配，必须在集合 $N(P, j)$ 中挑选最大的 t 。也就是说，当有多个值得试探的右移方案时，应该保守地选择其中移动距离最短者。于是，只需令

$$\text{next}[j] = \max(N(P, j))$$

则一旦发现 $P[j]$ 与 $T[i]$ 失配，就可以转而用 $P[\text{next}[j]]$ 与 $T[i]$ 继续比对。

既然集合 $N(P, j)$ 仅取决于模式串 P 以及失配位置 j ，而与主串无关，作为该集合内的最大者， $\text{next}[j]$ 也同样具有这一性质。于是，对于任一模式串 P ，不妨通过预处理提前计算出所有位置 j 所对应的 $\text{next}[j]$ 值，并整理为表格以便此后反复查询——亦即，将“记忆力”转化为“预知力”。

11.3.3 KMP算法

上述思路可整理为代码11.3，即著名的KMP算法^②。

这里，假定可通过 $\text{buildNext}()$ 构造出模式串 P 的 next 表。对照代码11.1的蛮力算法，只是在 else 分支对失配情况的处理手法有所不同，这也是KMP算法的精髓所在。

^② Knuth和Pratt师徒，与Morris几乎同时发明了这一算法。他们稍后联合署名发表^[60]该算法，并以其姓氏首字母命名

```
1 int match(char* P, char* T) { //KMP算法
2     int* next = buildNext(P); //构造next表
3     int n = (int) strlen(T), i = 0; //主串指针
4     int m = (int) strlen(P), j = 0; //模式串指针
5     while ((j < m) && (i < n)) //自左向右逐个比对字符
6         if (0 > j || T[i] == P[j]) //若匹配, 或P已移出最左侧 (两个判断的次序不可交换)
7             { i++; j++; } //则转到下一字符
8         else //否则
9             j = next[j]; //模式串右移 (注意: 主串不用回退)
10    delete [] next; //释放next表
11    return i - j;
12 }
```

代码11.3 KMP主算法

11.3.4 next[0] = -1

空串是任何非空串的真子串、真前缀和真后缀，故只要 $j > 0$ 则必有 $\emptyset \in N(P, j)$ 。此时的 $N(P, j)$ 必非空，从而保证“在其中取最大值”这一操作的确可行。但反过来，若 $j = 0$ ，则前缀 $\text{prefix}(P, j)$ 本身就是空串，它没有真子串，于是必有集合 $N(P, j) = \emptyset$ 。

此种情况下，又该如何定义 $\text{next}[0]$ 呢？按照串匹配算法的构思，若某轮迭代中首对字符即失配，则应将模式串P直接右移一个字符，然后从其首字符起继续下一轮比对。

表11.3 next表实例：假想地附加一个通配符P[-1]

rank	-1	0	1	2	3	4	5	6	7	8	9
P[]	*	C	H	I	N	C	H	I	L	L	A
next[]	N/A	-1	0	0	0	0	1	2	3	0	0

如表11.3所示，不妨假想地在 $P[0]$ 的左侧“附加”一个 $P[-1]$ ，而且该字符与任何字符都是匹配的。于是就实际效果而言，上述处理方法完全等同于“令 $\text{next}[0] = -1$ ”。

11.3.5 next[j + 1]

那么，若已知 $\text{next}[0, j]$ ，如何才能递推地计算出 $\text{next}[j + 1]$ ？是否有高效方法？

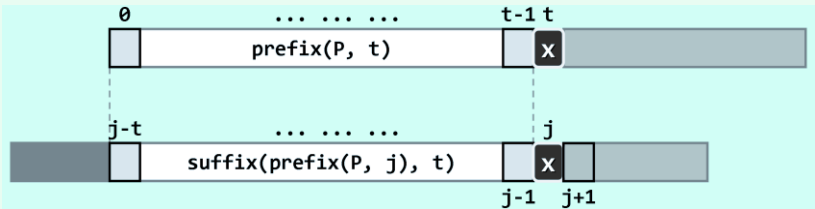


图11.7 $P[j] = P[\text{next}[j]]$ 时，必有 $\text{next}[j + 1] = \text{next}[j] + 1$

若 $\text{next}[j] = t$ ，则意味着在前缀 $\text{prefix}(P, j)$ 中，自匹配的真前缀和真后缀的最大长度为 t ，故必有 $\text{next}[j + 1] \leq \text{next}[j] + 1$ ——而且特别地，当且仅当 $P[j] = P[t]$ 时如图11.7取等号。那么一般地，若 $P[j] \neq P[t]$ ，又该如何得到 $\text{next}[j + 1]$ ？

此种情况下如图11.8, 由next表的定义, $\text{next}[j + 1]$ 的下一候选者应该依次是 $\text{next}[\text{next}[j]] + 1, \text{next}[\text{next}[\text{next}[j]]] + 1, \dots$

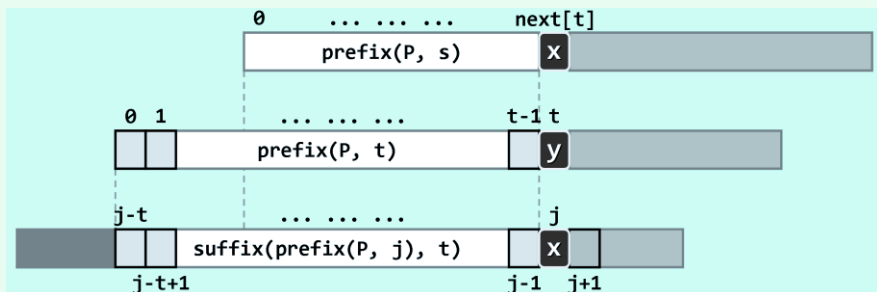


图11.8 $P[j] \neq P[\text{next}[j]]$ 时, 必有 $\text{next}[j + 1] = \text{next}[\dots \text{next}[j] \dots] + 1$

因此, 只需反复用 $\text{next}[t]$ 替换 t (即令 $t = \text{next}[t]$), 即可按优先次序遍历以上候选者; 一旦发现 $P[j]$ 与 $P[t]$ 匹配 (含通配), 即可将 $\text{next}[t] + 1$ 赋予 $\text{next}[j + 1]$ 。

既然总有 $\text{next}[t] < t$, 故在此过程中 t 必然严格递减; 同时, 即便 t 降低至0, 亦必然会终止于通配的 $\text{next}[0] = -1$, 而不致下溢。如此, 该算法的正确性完全可以保证。

11.3.6 构造next表

按照以上思路, 可实现next表构造算法如代码11.4所示。

```
1 int* buildNext(char* P) { //构造模式串P的next表
2     size_t m = strlen(P), j = 0; // “主” 串指针
3     int* N = new int[m]; //next表
4     int t = N[0] = -1; //模式串指针
5     while (j < m - 1)
6         if (0 > t || P[j] == P[t]) { //匹配
7             j++; t++;
8             N[j] = t; //此句可改进...
9         } else //失配
10            t = N[t];
11     return N;
12 }
```

代码11.4 next表的构造

可见, next表的构造算法与KMP算法几乎完全一致。实际上按照以上分析, 这一构造过程完全等效于模式串的自我匹配, 因此两个算法在形式上的近似亦不足为怪。

11.3.7 性能分析

■ $O(nm)$?

通过上述分析与实例可以看出, 相对于蛮力算法, KMP算法 (代码11.3) 借助next表可避免大量不必要的字符比对操作。然而就渐进意义而言, 时间复杂度会有实质性的改进吗? 这一点并非一目了然, 甚至乍看起来并不乐观。比如, 从算法流程的角度来看, 该算法依然需做 $O(n)$ 轮

迭代，而且任何一轮迭代都有可能需要比对多达 $\Omega(m)$ 对字符。

如此说来，在最坏情况下，KMP算法仍有可能共需执行 $\mathcal{O}(nm)$ 次比对？不是的。正如以下更为精确的分析将要表明的，即便在最坏情况下，KMP算法也只需运行线性的时间！

■ $\mathcal{O}(n)!$

为此，请注意代码11.3中用作字符指针的变量*i*和*j*。若令 $k = 2i - j$ 并考查*k*在KMP算法过程中的变化趋势，则不难发现：**while**循环每迭代一轮，*k*都会严格递增。

为验证这一点，只需分别核对**while**循环内部的**if-else**分支。无非两种情况：若执行**if**分支，则*i*和*j*同时加一，于是 $k = 2i - j$ 必将增加；反之若执行**else**分支，则尽管*i*保持不变，但在赋值 $j = \text{next}[j]$ 之后*j*必然减小，于是 $k = 2i - j$ 也必然会增加。

现在，纵观算法的整个过程：启动时有 $i = j = 0$ ，即 $k = 0$ ；算法结束时 $i \leq n$ 且 $j \geq 0$ ，故有 $k \leq 2n$ 。也就是说，在此期间尽管*k*从0开始持续地严格单调递增，但总体增幅不超过2*n*。既然*k*为整数，故**while**循环至多执行2*n*轮。另外，**while**循环体内部不含任何循环或调用，故只需 $\mathcal{O}(1)$ 时间。因此，若不计构造next表所需的时间，KMP算法的运行时间不超过 $\mathcal{O}(n)$ ——这一结论也可等效地理解为，KMP算法单步迭代的分摊复杂度仅为 $\mathcal{O}(1)$ 。

■ 总体复杂度

既然next表构造算法的流程与KMP算法并无实质区别，故仿照上述分析可知，next表的构造仅需 $\mathcal{O}(m)$ 时间。综上可知，KMP算法的总体运行时间为 $\mathcal{O}(n + m)$ 。

11.3.8 继续改进

尽管以上KMP算法已可保证线性的运行时间，但在某些情况下仍有进一步改进的余地。

■ 反例

考查模式串 $P = "000010"$ 。按照11.3.2节的定义，其next表应如表11.4所示。

在KMP算法过程中，假设如图11.9前一轮比对因 $T[i] = '1' \neq '0' = P[3]$ 失配而中断。于是按照以上的next表，接下来KMP算法将依次将 $P[2]$ 、 $P[1]$ 和 $P[0]$ 与 $T[i]$ 对准并做比对。

表11.4 next表仍有待优化的实例

rank	-1	0	1	2	3	4	5
P[]	*	0	0	0	0	1	0
next[]	N/A	-1	0	1	2	3	0

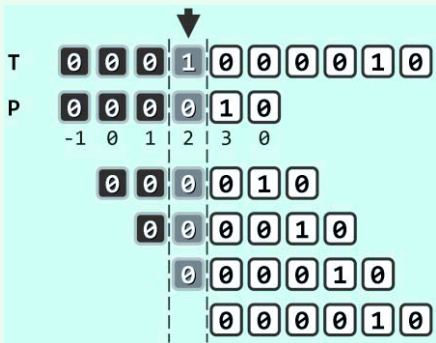


图11.9 按照此前定义的next表，仍有可能进行多次本不必要的字符比对操作

从图11.9可见，这三次比对都报告“失配”。那么，这三次比对的失败结果属于偶然吗？进一步地，这些比对能否避免？

实际上，即便说 $P[3]$ 与 $T[i]$ 的比对还算必要，后续的这三次比对却都是不必要的。实际上，它们的失败结果早已注定。

只需注意到 $P[3] = P[2] = P[1] = P[0] = '0'$ ，就不难看出这一点——既然经过此前的比对已发现 $T[i] \neq P[3]$ ，那么继续将 $T[i]$ 和那些与 $P[3]$ 相同的字符做比对，既重蹈覆辙，更徒劳无益。

■ 记忆 = 教训 = 预知力

从算法策略的层次来看，11.3.2节引入next表的实质作用，在于帮助我们利用以往成功比对所提供的经验，将记忆力转化为预知力。然而实际上，此前已进行过的比对，还远不止这些。确切地说，还包括那些失败的比对——作为“教训”，它们同样有益，但可惜此前一直被忽略了。

依然以图11.9为例，以往的失败比对，实际上已经为我们提供了一条极为重要的信息—— $T[i] \neq P[4]$ ——可惜我们却未能有效地加以利用。原算法之所以会执行后续四次本不必要的比对，原因也正在于未能充分汲取教训。

■ 改进

为把这类“负面”信息引入next表，只需将11.3.2节中集合 $N(P, j)$ 的定义修改为：

$$N(P, j) = \{ t \mid \text{prefix}(\text{prefix}(P, j), t) = \text{suffix}(\text{prefix}(P, j), t) \\ \text{且 } P[j] \neq P[t], 0 \leq t < j \}$$

也就是说，除“对应于自匹配长度”以外， t 只有还同时满足“当前字符对不匹配”的必要条件，方能归入集合 $N(P, j)$ 并作为next表项的候选。

相应地，原next表构造算法（代码11.4）也需稍作修改，调整为如下改进版本。

```
1 int* buildNext(char* P) { //构造模式串P的next表（改进版本）
2     size_t m = strlen(P), j = 0; // “主”串指针
3     int* N = new int[m]; //next表
4     int t = N[0] = -1; //模式串指针
5     while (j < m - 1)
6         if (0 > t || P[j] == P[t]) { //匹配
7             j++; t++;
8             N[j] = (P[j] != P[t]) ? t : N[t]; //注意此句与未改进之前的区别
9         } else //失配
10            t = N[t];
11     return N;
12 }
```

代码11.5 改进的next表构造算法

由代码11.5可见，改进后的算法与原算法的唯一区别在于，每次在 $\text{prefix}(P, j)$ 中发现长度为 t 的真前缀和真后缀相互匹配之后，还需进一步检查 $P[j]$ 是否等于 $P[t]$ 。唯有在 $P[j] \neq P[t]$ 时，才能将 t 赋予 $\text{next}[j]$ ；否则，需转而代之以 $\text{next}[t]$ 。

仿照11.3.7节的分析方法易知，改进后next表的构造算法同样只需 $O(m)$ 时间。

■ 实例

仍以 $P = "000010"$ 为例，改进之后的next表如表11.5所示。读者可参照图11.9，就计算效率将新版本与原版本（表11.4）做一对比。

表11.5 改进后的next表实例

rank	-1	0	1	2	3	4	5
P[j]	*	0	0	0	0	1	0
next[j]	N/A	-1	-1	-1	-1	3	-1

利用新的next表针对图11.9中实例重新执行KMP算法，在首轮比对因 $T[i] = '1' \neq '0' = P[3]$ 失配而中断之后，将随即以 $P[\text{next}[3]] = P[-1]$ （虚拟通配符）与 $T[i]$ 对齐，并启动下一轮比对。将其效果而言，等同于聪明且安全地跳过了三个不必要的对齐位置。

§ 11.4 *BM算法

11.4.1 思路与框架

■ 构思

KMP算法的思路可概括为：当前比对一旦失配，即利用此前的比对（无论成功或失败）所提供的信息，尽可能长距离地移动模式串。其精妙之处在于，无需显式地反复保存或更新比对的历史，而是独立于具体的主串，事先根据模式串预测出所有可能出现的失配情况，并将这些信息“浓缩”为一张next表。就其总体思路而言，本节将要介绍的BM算法^③与KMP算法类似，二者的区别仅在于预测和利用“历史”信息的具体策略与方法。

BM算法中，模式串P与主串T的对准位置依然“自左向右”推移，而在每一对准位置却是“自右向左”地逐一比对各字符。具体地，在每一轮自右向左的比对过程中，一旦发现失配，则将P右移一定距离并再次与T对准，然后重新一轮自右向左的扫描比对。为实现高效率，BM算法同样需要充分利用以往的比对所提供的信息，使得P可以“安全地”向后移动尽可能远的距离。

■ 主体框架

BM算法的主体框架，可实现如代码11.6所示。

```
1 int match(char* P, char* T) { //Boyer-Morre算法（完全版，兼顾Bad Character与Good Suffix）
2     int* bc = buildBC(P); int* gs = buildGS(P); //构造BC表和GS表
3     size_t i = 0; //模式串相对于主串的起始位置（初始时与主串左对齐）
4     while (strlen(T) >= i + strlen(P)) { //不断右移（距离可能不止一个字符）模式串
5         int j = strlen(P) - 1; //从模式串最末尾的字符开始
6         while (P[j] == T[i + j]) //自右向左比对
7             if (0 > --j) break;
8         if (0 > j) //若极大匹配后缀 == 整个模式串（说明已经完全匹配）
9             break; //返回匹配位置
10        else //否则，适当地移动模式串
11            i += __max(gs[j], j - bc[ T[i + j] ]); //位移量根据BC表和GS表选择大者
12    }
13    delete [] gs; delete [] bc; //销毁GS表和BC表
14    return i;
15 }
```

代码11.6 BM主算法

可见，这里采用了蛮力算法后一版本（310页代码11.2）的方式，借助整数i和j指示主串中当前的对齐位置T[i]和模式串中接受比对的字符P[j]。不过，一旦局部失配，这里不再是机械地令i += 1并在下一字符处重新对齐，而是采用了两种启发式策略确定最大的安全移动距离。为此，需经过预处理，根据模式串P整理出坏字符和好后缀两类信息。

与KMP一样，算法过程中指针i始终单调递增；相应地，P相对于T的位置也绝不后退。

^③ 由R. S. Boyer和J. S. Moore于1977年发明^[61]